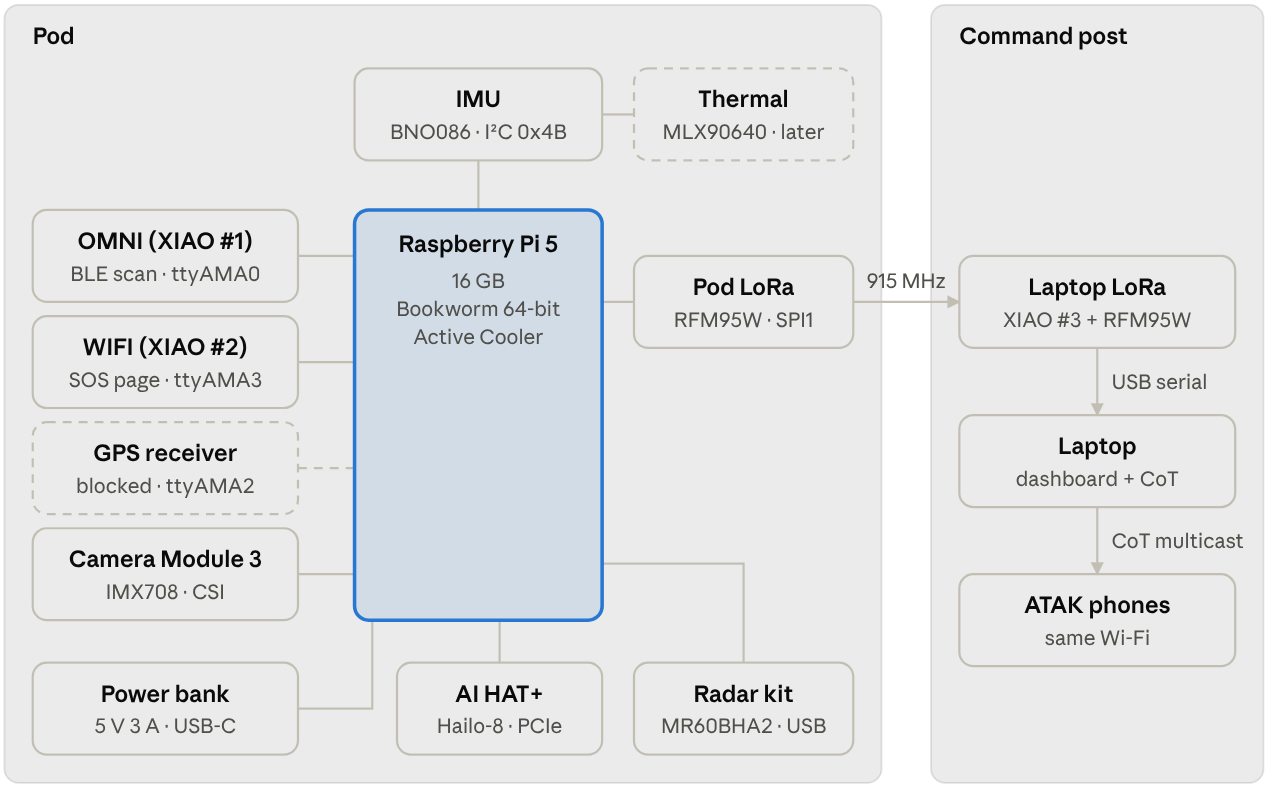
AFTERGLOW Pod Hardware Reference for Code

Nov 7, 2026 · @Rohan

System map

The pod is a Raspberry Pi 5 with eight boards around it; the command post is a laptop with the second LoRa radio. Dashed boxes aren't connected yet. Pinouts, ports and libraries for each board follow below.

Every pod board hangs off the Pi; alerts reach rangers by LoRa



pod and command post · how every board connects

Sighting flow: OMNI hears a beacon → Pi stamps time, position and distance from trail → pod LoRa sends an alert → laptop LoRa prints it over USB → dashboard pin and ATAK marker.

Raspberry Pi 5 (pod brain)

The Pi 5 talks to every pod board: two UARTs to the XIAOs, I²C to the IMU and thermal camera, SPI1 to the LoRa radio, USB to the radar, CSI to the camera and PCIe to the AI HAT+.

Item	Value
Board	Raspberry Pi 5, 16 GB, Active Cooler on the FAN header
OS	Raspberry Pi OS Bookworm 64-bit, `sudo apt full-upgrade`, then `sudo apt install hailo-all`
Power	USB-C power bank, 5 V 3 A or more. No drone power.
GPIO in Python	Use `gpiozero` / `lgpio` (or Blinka). `RPi.GPIO` does not work on a Pi 5.
SPI0	Leave **off** (`dtparam=spi=on` must not be set): its GPIO8/9 are WIFI's UART3

Add to `/boot/firmware/config.txt`, then reboot:

```
dtparam=i2c_arm=on
dtparam=i2c_arm_baudrate=400000   # BNO086 + MLX90640 bus
dtparam=uart0=on                  # GPIO14/15 -> /dev/ttyAMA0 (OMNI)
dtoverlay=uart3-pi5               # GPIO8/9   -> /dev/ttyAMA3 (WIFI)
dtoverlay=uart2-pi5               # GPIO4/5   -> /dev/ttyAMA2 (GPS, once it
exists)
dtoverlay=spi1-1cs                # SPI1      -> /dev/spidev1.0 (pod LoRa)
usb_max_current_enable=1          # power bank can't negotiate 5 A
camera_auto_detect=1
```

In `raspi-config` → Interface → Serial port: login shell **No**, serial hardware **Yes**. On a Pi 5 the login console lives on its own debug port (`/dev/ttyAMA10`), so UART0 stays free.

Device node	What it is
`/dev/ttyAMA0`	OMNI XIAO, 921600 8N1

Device node	What it is
`/dev/ttyAMA3`	WIFI XIAO, 921600 8N1
`/dev/ttyAMA2`	GPS receiver (reserved, not wired yet)
`/dev/i2c-1`	IMU 0x4B, thermal 0x33 (later)
`/dev/spidev1.0`	Pod LoRa RFM95W
`/dev/ttyACM0`	Radar kit over USB (the only USB device)
`/dev/video*`, libcamera	Camera Module 3
`/dev/hailo0`	AI HAT+

Pi header pin map

25 of the 40 pins are in use. Pins 27 and 28 belong to the AI HAT+ ID chip and must stay empty. All logic is 3.3 V.

Pin	GPIO / rail	Function	Goes to	Code uses
1	3.3 V	Power	IMU 3V3 (thermal via Qwiic later)	—
2	5 V	Power	OMNI 5V + WIFI 5V (Y-lead)	—
3	GPIO2	I²C1 SDA	IMU SDA	`/dev/i2c-1`
4	5 V	Power	GPS VCC (reserved)	—
5	GPIO3	I²C1 SCL	IMU SCL	`/dev/i2c-1`
6	GND	Ground	IMU GND	—
7	GPIO4	UART2 TX	GPS RX (reserved)	`/dev/ttyAMA2`
8	GPIO14	UART0 TX	OMNI D9 (RX)	`/dev/ttyAMA0`
9	GND	Ground	OMNI GND	—
10	GPIO15	UART0 RX	OMNI D10 (TX)	`/dev/ttyAMA0`
12	GPIO18	SPI1 CE0	LoRa CS	`/dev/spidev1.0`
14	GND	Ground	WIFI GND	—

Pin	GPIO / rail	Function	Goes to	Code uses
15	GPIO22	Input	IMU INT (active low)	GPIO22
16	GPIO23	Output	IMU RST (active low)	GPIO23
17	3.3 V	Power	LoRa VIN	—
18	GPIO24	Input	LoRa G0 / DIO0 (interrupt)	GPIO24
21	GPIO9	UART3 RX	WIFI D10 (TX)	`/dev/ttyAMA3`
22	GPIO25	Output	LoRa RST (active low)	GPIO25
24	GPIO8	UART3 TX	WIFI D9 (RX)	`/dev/ttyAMA3`
29	GPIO5	UART2 RX	GPS TX (reserved)	`/dev/ttyAMA2`
30	GND	Ground	GPS GND (reserved)	—
35	GPIO19	SPI1 MISO	LoRa MISO	`/dev/spidev1.0`
38	GPIO20	SPI1 MOSI	LoRa MOSI	`/dev/spidev1.0`
39	GND	Ground	LoRa GND	—
40	GPIO21	SPI1 SCLK	LoRa SCK	`/dev/spidev1.0`
27, 28	ID_SD / ID_SC	HAT EEPROM	AI HAT+ — do not use	—

Free for later: GPIO6, 7, 10, 11, 12, 13, 16, 17, 26, 27 (pins 31, 26, 19, 23, 32, 33, 36, 11, 37, 13).

OMNI: Bluetooth scanner (XIAO ESP32S3 #1)

OMNI scans Bluetooth LE for Find My beacons and streams each sighting to the Pi as one JSON line over UART0. D6 and D7 are not used.

Item	Value
Board	Seeed XIAO ESP32S3, Arduino core for ESP32 (board: "XIAO_ESP32S3")
Power	5V pad from Pi pin 2. Unplug that wire before flashing over USB-C.

Item	Value
Antenna	Stick-on antenna on the U.FL socket, left wall. Without it the board barely hears.
UART to Pi	`Serial1.begin(921600, SERIAL_8N1, D9, D10);` — receive on D9 (GPIO8), transmit on D10 (GPIO9)
Pi side	`/dev/ttyAMA0`, 921600 8N1, no flow control
Debug	`Serial` (USB CDC) stays free for logs while on the bench

What to scan for. Find My beacons are BLE advertisements with Apple's company ID `0x004C` followed by type `0x12`. A device near its owner sends a short payload (length `0x02`). A device away from its owner (lost AirTag, powered-off iPhone) sends the full 25-byte payload (length `0x19`). That full payload is the "separated" filter. The advertising address rotates: about every 15 minutes near the owner, about once a day when separated. Verify all of this against a real AirTag on Saturday before trusting it.

Proposed line format (agree it with the Pi code before writing either side):

```
{"t":123456,"mac":"d4:3a:..","rssi":-71,"type":"findmy","sep":true,"key":"
<hex of payload>"}
```

One object per line, `\n` terminated. `t` = OMNI's `millis()`; the Pi stamps its own time and position on arrival.

WIFI: SOS page (XIAO ESP32S3 #2)

WIFI runs a Wi-Fi access point with a page that tells a lost person help is coming. The Pi switches it on and off and sets the message over UART3. Same board and pins as OMNI.

Item	Value
Board	Seeed XIAO ESP32S3
Power	5V pad from the other leg of the pin-2 Y-lead
Antenna	Stick-on antenna on the U.FL socket, right wall, as far from OMNI's as possible

Item	Value
UART to Pi	`Serial1.begin(921600, SERIAL_8N1, D9, D10);` — D9 receive, D10 transmit
Pi side	`/dev/ttyAMA3` (UART3, GPIO8 TX / GPIO9 RX), 921600 8N1
Likely code	`WiFi.softAP()` + `DNSServer` (answer every name with the board's IP so phones show the page) + `WebServer`

Proposed commands from the Pi, one JSON line each: `{"cmd":"sos","on":true,"msg":"Help is coming. Stay where you are."}` and `{"cmd":"sos","on":false}`. WIFI replies `{"ok":true}` and can report `{"evt":"client","n":1}` when a phone joins, which is itself a sign someone is there.

IMU: SparkFun BNO086

The IMU gives the pod's orientation, so a reading can be tied to which way the camera and antennas pointed. It sits at I²C address **0x4B** with reset and interrupt wired to Pi GPIOs.

Item	Value
Board	SparkFun VR IMU Breakout BNO086 (Qwiic), SEN-22857
Bus	`/dev/i2c-1`, 400 kHz, 2.2 kΩ pull-ups on the board
Address	0x4B (ADR jumper open, as wired). 0x4A only if ADR is bridged.
Power	3.3 V only, from Pi pin 1. Never 5 V.
INT	GPIO22 (pin 15), active low: goes low when data is ready
RST	GPIO23 (pin 16), active low: pulse low to reset
Python	`adafruit-circuitpython-bno08x` via Blinka; the BNO086 speaks the same SH-2 protocol as the BNO085

Item	Value
Useful reports	rotation vector (quaternion), game rotation vector, accelerometer, gyro, magnetometer

Starting point: `BNO08X_I2C(board.I2C(), address=0x4B, reset=DigitalInOut(board.D23))`. SparkFun's own notes say the BNO086 only runs reliably with RST and INT connected, which is why they're wired. If I²C reads fail intermittently, try dropping the bus to 100 kHz.

Thermal camera: MLX90640 (arriving later)

The thermal camera is a shared item and will join the same I²C bus later through a Qwiic cable from the IMU. It needs no new Pi pins.

Item	Value
Sensor	MLX90640, 32 × 24 pixels. Breakout make and lens angle not known yet.
Bus	`/dev/i2c-1`, shared with the IMU
Address	0x33
Python	`adafruit-circuitpython-mlx90640`
Frame rate	Set with `refresh_rate`; at 400 kHz expect only a few frames a second

Use it to confirm a warm body at a pin, not to search: 768 pixels is coarse.

Pod LoRa radio: Adafruit RFM95W on Pi SPI1

The pod's radio is a bare RFM95W (Semtech SX1276 inside) driven by the Pi over SPI1. It sends alerts to its twin at the laptop. Meshtastic is out: these are bare radios, so both ends send plain LoRa packets.

Signal	Pi pin	GPIO	Radio pad
Power	17	3.3 V	VIN (board accepts 3.3–6 V)

Signal	Pi pin	GPIO	Radio pad
Ground	39	GND	GND
Clock	40	GPIO21 SPI1 SCLK	SCK
Pi → radio	38	GPIO20 SPI1 MOSI	MOSI
Radio → Pi	35	GPIO19 SPI1 MISO	MISO
Chip select	12	GPIO18 SPI1 CE0	CS
Reset	22	GPIO25	RST
Interrupt	18	GPIO24	G0 (DIO0)
—	—	—	EN: unconnected (board on)

Library. `adafruit-circuitpython-rfm9x` via Blinka: `busio.SPI(board.SCK_1, MOSI=board.MOSI_1, MISO=board.MISO_1)`, CS = `D18`, reset = `D25`. It speaks the RadioHead packet format, so the laptop side should use RadioHead's `RH_RF95` to match.

Chip-select clash. `spi1-1cs` hands GPIO18 to the kernel as CE0, while the Adafruit library wants to drive CS itself. If you get "GPIO busy", move the kernel's CS out of the way with `dtoverlay=spi1-1cs,cs0_pin=16` (GPIO16 is free) and keep CS on GPIO18 in code.

Radio settings: both ends must match exactly.

Setting	Value
Frequency	915.0 MHz (US band)
Spreading factor	7 to start; 9–10 for more range at lower speed
Bandwidth	125 kHz
Coding rate	4/5
Preamble	8 symbols
TX power	Start at 13 dBm, up to 20 dBm
Max payload	251 bytes after RadioHead's 4-byte header

The antenna is the 915 MHz spring antenna soldered into **ANT**. Never transmit without it. Proposed alert: a compact JSON line under ~60 bytes, e.g. `{"id":7,"lat":44.35,"lon":-68.22,"off":42,"rssi":-71}`.

Laptop LoRa radio: XIAO ESP32S3 #3 + RFM95W

The second RFM95W is wired to XIAO #3, which plugs into the laptop by USB. It receives pod alerts and prints each one to USB serial for `lora_rx.py`. This is the last XIAO; there's no spare.

XIAO pad	ESP32-S3 GPIO	Radio pad	Arduino name
5V	USB 5 V	VIN	—
GND	GND	GND	—
D8	GPIO7	SCK	`SCK` (default SPI)
D9	GPIO8	MISO	`MISO` (default SPI)
D10	GPIO9	MOSI	`MOSI` (default SPI)
D0	GPIO1	CS	`D0`
D1	GPIO2	RST	`D1`
D3	GPIO4	G0 (DIO0)	`D3`

D6 and D7 are unused.

Firmware sketch outline (Arduino, RadioHead):

```
#include <SPI.h>
#include <RH_RF95.h>
RH_RF95 rf95(D0, D3);            // CS, interrupt
void setup() {
  Serial.begin(115200);           // USB CDC to the laptop
  pinMode(D1, OUTPUT);            // reset pulse
  digitalWrite(D1, LOW); delay(10); digitalWrite(D1, HIGH); delay(10);
  rf95.init(); rf95.setFrequency(915.0); rf95.setTxPower(13, false);
}
void loop() {
  if (rf95.available()) {
    uint8_t buf[RH_RF95_MAX_MESSAGE_LEN]; uint8_t len = sizeof(buf);
    if (rf95.recv(buf, &len)) { Serial.write(buf, len); Serial.printf("
rssi=%d\n", rf95.lastRssi()); }
  }
}
```

Radio settings must match the pod radio's table above. On the laptop the board shows up as `/dev/ttyACM*` (macOS `/dev/cu.usbmodem*`, Windows `COMx`); read it with `pyserial` at 115200.

Camera Module 3 and AI HAT+

The camera feeds the AI HAT+, which runs person detection at the pin so the ranger sees a box around anyone in view.

Item	Value
Camera	Raspberry Pi Camera Module 3 (Sony IMX708, 12 MP, autofocus) on a CAM/DISP port, Pi 5 22-to-15-pin cable
Camera software	`libcamera` / `rpicam-apps` (`rpicam-hello --list-cameras` to check), Python `picamera2`
AI accelerator	Raspberry Pi AI HAT+ 26 TOPS (Hailo-8) on the Pi's PCIe connector
AI software	`hailo-all` package; check with `hailortcli fw-control identify`

Item	Value
Models	Pre-compiled `.hef` files, e.g. a YOLO person detector from the Hailo Raspberry Pi 5 examples
Header	The HAT passes all 40 pins through its stacking header, but keeps pins 27/28 for its ID chip

Radar kit: Seeed MR60BHA2

The radar reads breathing, heart rate and presence at short range. It's a full kit with its own XIAO ESP32C6, and plugs into the Pi by USB.

Item	Value
Kit	Seeed MR60BHA2: 60 GHz radar + XIAO ESP32C6 + BH1750 light sensor + WS2812 LED
Shipped firmware	ESPHome (for Home Assistant). **Reflash** the XIAO ESP32C6 with an Arduino sketch to get data over USB.
Library	Seeed mmWave library, `#include <Seeed_Arduino_mmWave.h>`, class `SEEED_MR60BHA2`
Calls	`update(100)`, `getBreathRate(float&)`, `getHeartRate(float&)`, `getDistance(float&)`, `isHumanDetected()`
Link to Pi	USB-C to a black USB 2 port; appears as `/dev/ttyACM0`. Print JSON lines at 115200.
Note	Updating the radar module's own firmware wipes the XIAO's sketch; reflash after.

Proposed line: `{"breath":14.2,"heart":78,"dist":0.9,"human":true}`.

GPS (blocked: receiver board needed)

There is no GPS position yet. The team has a u-blox ANN-MB5 antenna (L1/L5, SMA, 3 m cable, ~104 g), which is only an antenna; it needs a receiver board to screw into. Write position code against an interface so the source can change.

Item	Value
Reserved pins	4 (5 V), 7 (GPIO4 TX2), 29 (GPIO5 RX2), 30 (GND)
Port	`/dev/ttyAMA2` once wired; baud = the receiver's default (u-blox M10/F10 parts are usually 38400)
Data	NMEA sentences (`$GNGGA` , `$GNRMC`); parse with `pynmea2` , or run `gpsd`
If the receiver has Qwiic	It can sit on `/dev/i2c-1` instead (u-blox default address 0x42) and free the four UART pins
Indoor demo	No GPS indoors. Use taped waypoints mapped to fake coordinates, or replay a recorded track.

Suggested shape: `position.py` exposes `get_fix() -> (lat, lon, source)` with `source` = `gps` , `waypoint` or `replay` .

Laptop, dashboard and ATAK

The laptop is the command post: it reads alerts from the laptop LoRa radio, shows them on the dashboard and forwards them to rangers' phones in ATAK.

Piece	Detail
`lora_rx.py`	`pyserial` on the XIAO #3 port at 115200; parse one alert per line
Dashboard	Map with trails, campground zone, pins; buttons per pin: acknowledge, investigate, clear
`cot_out.py`	Sends Cursor-on-Target XML over UDP multicast to `239.2.3.1:6969` on the laptop's Wi-Fi
ATAK	Phones on the same Wi-Fi see each pin as a marker

No cellular or internet is needed anywhere in the chain.

Bring-up checks and open questions

Run these on the Pi after the config.txt edits; each line names what success looks like.

- [ ] `ls /dev/ttyAMA*` shows `ttyAMA0` and `ttyAMA3` (and `ttyAMA2` once the GPS is in)
- [ ] `ls /dev/spidev*` shows `spidev1.0`
- [ ] `i2cdetect -y 1` shows `4b` (and `33` once the thermal camera is in)
- [ ] `rpicam-hello --list-cameras` lists the IMX708
- [ ] `hailortcli fw-control identify` finds the Hailo-8
- [ ] `ls /dev/ttyACM*` shows the radar
- [ ] `vcgencmd get_throttled` reads `0x0` with everything running on the power bank
- [ ] A packet sent from the pod radio prints on the laptop radio's serial port

Open questions:

- [ ] Does the Find My "separated" test (type `0x12`, length `0x19`) hold on our own AirTag and a powered-off iPhone?
- [ ] Which GPS receiver board, and does it have Qwiic?
- [ ] Which MLX90640 breakout (lens angle) is the shared thermal camera?
- [ ] Agree the line formats for OMNI, WIFI, radar and LoRa alerts before both sides are coded
- [ ] Does the pod still fly on a drone, or only ride the pole?

Sources

- SparkFun BNO086 hardware overview
- SparkFun forum: BNO086 needs RST and INT
- Seeed MR60BHA2 getting started
- Hat Labs: Pi 5 UART overlays and ports
- Core Electronics: u-blox ANN-MB5
- Pod Bench Wiring page, rev F (team artifact)